

Obolus: An Agent-Native Micropayment Rail for Human Evidence over HTTP 402 on Solana

Team SAIL

Jihun Yun

Kyuhyun Ro

Yuchan Lee

August 2026

Abstract

Autonomous agents can already read the public web for free, but they cannot buy a single specific answer from the person who knows it. Obolus is a search service that reads documents people wrote themselves instead of the web and settles for them *one at a time* over HTTP 402. Free search returns only handles and prices, never a passage; a query-bound, immutable quote is then paid over the **x402 v2 exact/SVM** scheme in **Solana Devnet USDC**, and the exact passage is released only against that receipt. The system is deployed production-shaped: a Cloudflare Pages edge over **four independent Cloud Run services** (core API, x402 gateway, payment/recovery worker, and a token-gated Pay.sh front) in Google Cloud, backed by Cloud SQL PostgreSQL 16, Cloud Tasks, a create-only audit prefix, and a non-exportable Ed25519 signer in Cloud KMS. The custodial ledger is denominated in **USDC atomic units** (six decimals) end to end; a standalone prepaid balance tops up over the same 402 exchange for whole-USDC amounts and is credited *only after two independent RPC providers agree the transfer is finalized*, idempotent by transaction signature. No user key, browser-helper key, or SPL delegate ever reaches the backend — the service wallet signs through KMS alone. The design principle is **payment fidelity**: every quote, split, refund, and recovery is exact and server-authoritative, and the AI plans and synthesizes but never spends or becomes priced inventory. We report the deployed architecture, an honest measured-hardening record of four money-path defects we found and fixed, and the concrete gates that remain before Mainnet or paid customers. Verified live 2026-08-20: the API and gateway serve 100% of traffic from the current main revision, and an end-to-end wallet → top-up → balance run passes against production on both the direct and same-origin-proxy surfaces.

1 Introduction

The web is a corpus an agent reads for free, and that is exactly its limit: the answer a practitioner would give — the migration that actually worked, the number that was really measured — is either not written down or not for sale one unit at a time. Advertising and subscriptions price attention and access in bulk; neither prices a *single* document at the moment an agent needs it. Micropayments have been technically possible for years, but the missing piece was never the transfer — it was a settlement contract an autonomous buyer can be trusted with: a fixed recipient, a fixed amount, a fixed asset, and a receipt that releases exactly what was paid for and nothing else.

Obolus is that contract, deployed. Its unit is the **human document** — one quality-checked, versioned final answer — and its rail is **HTTP 402** settled in Solana Devnet USDC. Search is free and returns handles and prices; a paid open delivers the immutable passage bound to that query and receipt. The buyer is a browser with a Phantom wallet or an autonomous agent behind an MCP surface, and in both cases the money moves under a template the buyer cannot rewrite. This report describes the system, the deterministic boundary that owns all money and identity, the two settlement paths, and — in the spirit of a build report

rather than a brochure — the four money-path defects we caught by measurement and the design decision each one forced.

2 System Overview

A request enters as free search and leaves as a settled, cited passage (Fig. 1). Rust owns the decision: matching, ranking, budget filtering, author deduplication, and the hit/miss branch all run in the core API over 768-dimensional hash embeddings with word and character n-grams, entity anchors, trust, freshness, and a topic-personalized PageRank over curator-verified evidence edges. Paid, self-owned, inferred, and raw edges cannot buy authority. The search response carries handles and prices and *never* a passage.

Hit and miss. On a hit, the server commits one job — exact handles, content hashes, beneficiary wallets, per-document atomic prices, total, mint, network, and expiry — reserves the cost from prepaid credit, settles each owner, and releases only the paid snapshots. On a miss the agent may take a free question-only baseline, then the flow inverts into the one dialogue the project had to invent: *"nobody has covered this yet — ask people? how many? at what price per answer?"* A funded open call is posted to a board, and later answers settle back to the asker. That state machine is the whole product surface; everything under it is exact accounting.

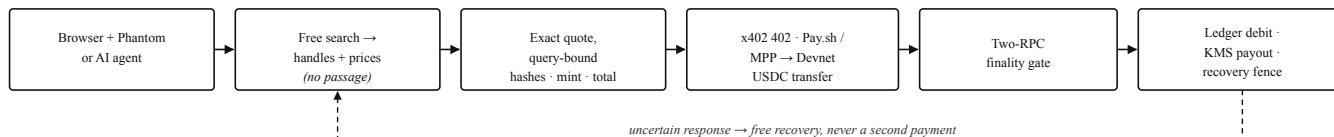


Figure 1: One settlement cycle. Free search returns only handles and prices; a query-bound, immutable quote is paid over x402 — a browser signs an exact refill in Phantom against prepaid credit, or an agent settles through the Pay.sh gateway. Release and owner payout happen only after two independent RPC origins agree the transfer is finalized, and an uncertain or duplicated response follows a free recovery token rather than paying again. Every hop is idempotent by transaction signature.

3 Payment Rail and Ledger

Exact settlement over HTTP 402. Obolus speaks x402 v2 in the exact/SVM scheme. A quote fixes the recipient, asset (Devnet USDC, mint `4zMM...DncDU`), network (`solana:EtWT...qa1`), amount, query, and expiry; the gateway accepts only a byte-identical signed template against that quote. Managed conversion uses a fixed test rate of **1 USDC = ₩1,350**, and each document rounds *up* independently to atomic units, $\text{ceil}(\text{priceKrw} \times 10^6 / 1350)$, so a bundle never under-charges. The hosted rail currently accepts fixed test price bands from ₩5 to ₩1,000.

Prepaid ledger. The custodial ledger is denominated in USDC atomic units (six decimals) and serialized as JSON strings so a browser cannot lose precision; balances, earnings, escrow, receipts, and coverage totals all read in USDC, and the legacy won columns are kept only as nullable history. A browser reserves exactly the quoted amount from prepaid credit and Phantom signs only a buyer-chosen refill when credit is low — unused credit stays withdrawable and the service cannot pull more. Credit can also be raised on its own: the browser asks the gateway for a whole-USDC quote (**1–1,000 USDC**), pays it over the same exact scheme, and the gateway posts the verified transfer to an internal, token-gated deposit route *only after two independent RPC providers agree the transfer is finalized*. The deposit is idempotent by transaction signature, so a retry credits once.

Split, refund, recovery. The product policy is a 90% owner / 10% protocol split with no checkout surcharge; the hosted Devnet endpoints currently send all-but-one atomic unit to the owner, so a same-receipt on-chain 90/10 split remains an explicit pre-Mainnet gate rather than a claimed take rate. A permanent partial failure restores the exact unpaid atomic remainder to prepaid credit, and a lost or duplicated response is resolved by a query-scoped recovery token rather than a second payment. Signer rotation deliberately fails readiness until every payout and refund owned by the old wallet has drained.

Wallet-only identity. Authentication is a wallet challenge (SIWX). Sessions are server-issued `HttpOnly`, `SameSite=Lax` cookies; a client-supplied user id is rejected, and email/password stays test-only behind a disabled flag. A `wallet_identities` row is a 1:1 proof of key control and is the wallet's canonical

owner for its prepaid ledger — a property §5 shows we had to enforce precisely.

4 Agent Interface

Agents reach Obolus through an MCP runtime (Antigravity or plain MCP) exposing free metadata tools only: search, price, and a free recovery check. The model never sees a raw URL, wallet, header, mint, or payment credential. To buy, the local buyer agent stores one exact intent and requires an interactive, one-time approval of its aggregate atomic amount; the model cannot substitute the recipient, mint, network, amount, or URL, and the Pay.sh wallet stays behind a handshake adapter. Settlement itself is deterministic: `pay curl` drives the 402/MPP exchange per document against a query-bound resource, and Rust re-validates the immutable quote, band, asset, network, query, handle, and runtime recipient before releasing a snapshot.

A bounded model loop. Inside the API, Gemini on Vertex AI runs a two-call loop: call one maps the authenticated question to bounded search metadata; Rust executes and ranks retrieval; call two observes only aggregate hit/partial/miss state and selects one reviewed, non-spending next action. Economic proposals stop for explicit user approval, and paid synthesis is a separate post-settlement call that receives only server-proven opened passages. AI output is liquidity, never priced inventory or authority — it cannot enter documents, memory, or settlement — and the three auditable trace labels are roles in one loop, not three agents or A2A.

5 Reliability and Hardening

Money bugs do not announce themselves as crashes; they render as a wrong number a user trusts. We therefore treat each money-path correction the way the venue would treat an ablation — a measurement that falsified a prior assumption and forced a specific design decision. Table 2 is that record: four defects found on the live Devnet system, each with the observation that exposed it and the invariant that replaced it. Two share a root cause worth stating plainly: *a custodial balance must be owned by the account that proved control of the wallet's key, and it must reflect only value that actually settled on chain*.

Ownership follows the key, not the balance. The prepaid ledger is keyed by wallet, but ownership had been allowed to drift to an earlier account while the wallet's own login — the holder of the 1:1 `wallet_identities` row — was refused at top-up with a 409. Gating the repair on a zero balance made it

worse: a wallet holding real USDC stayed locked out of its own ledger. The fix re-homes the ledger to the identity holder *regardless of balance*, because a fresh challenge signed by the wallet key is stronger proof than any prior owner row, and it opens no theft surface a key-holder does not already have on chain. A pay-out-profile account with no wallet identity still cannot take a funded wallet — one custodial ledger per wallet stays intact. The reclaim was reproduced live end to end on both the direct and same-origin-proxy surfaces.

Only on-chain value is real. A legacy sandbox signup credit had been rendered as a USDC balance, so a card read a phantom

Retired approach	Observed result	Invariant that replaced it
Signup credit rendered as USDC	Card read a phantom 74.07 USDC balance no deposit backed	Reconcile the ledger to on-chain-backed deposits only; headline reads the prepaid pot + accrued earnings
Re-home gated on <code>available==0</code>	A funded wallet's own login account was refused (409) at top-up	The wallet-key signature proves control; re-home the ledger to the identity holder regardless of balance
20 s finality, single public RPC	Spurious "finalized RPC views did not agree" while the transfer had already settled	40 s window requiring genuine agreement of two independent RPC origins
Headline = won-sandbox ledger + accrued	Earnings double-counted, real prepaid balance omitted	The internal ledger already folds accrued in; read the prepaid pot + accrued earnings once

Table 2: The hardening record — four money-path defects found on the live Devnet system, each measured, then closed by an invariant rather than a patch. The pattern across all four: a displayed balance must name value that actually settled on chain, owned by the account that proved the wallet key.

6 Serving and Deployment

The `pages.dev` hostname is only the edge. Cloudflare Pages serves the fingerprinted Vite build and two narrow same-origin proxies — `/api/*` preserves the path to the core API, `/x402/*` strips its prefix before streaming to the gateway — while **four independent Cloud Run services** in Google Cloud (project `sweetspot-ax`, region `asia-northeast3`) own application logic, payment authorization, orchestration, and the protected Pay.sh boundary. Each runtime has a dedicated, keyless service account with workload-scoped IAM. Table 1 collects the deployment and economic constants.

Trust boundaries. `obolus-api` (Rust/Axum) owns wallet sessions, search and ranking, immutable quotes, the database-backed custodial ledger, escrow, receipts, disputes, and the Gemini policy. `obolus-gateway` validates exact x402/MPP economics, requires independent finality, buffers paid content until the ledger accepts it, and records settlement through Cloud Tasks; it never holds a user key. `obolus-orchestrator` pays selected evidence deterministically and prepares KMS-backed payouts and refunds, fencing ambiguous attempts for reconciliation rather than signing twice. `obolus-pay` runs the official Pay.sh collector behind a private gateway token — a request without it receives a 404, and browsers and agents never see that token or private origin. The system of record is Cloud SQL PostgreSQL 16; a create-only Cloud Storage prefix holds economic intent written before any external payment; the signer is a non-exportable Ed25519 key in Cloud KMS.

74.07 USDC that no deposit backed. We reconciled the ledger to on-chain-backed deposits only and pointed the headline figure at the prepaid pot plus accrued earnings — dropping the internal won-sandbox ledger, which already folds accrued earnings in and would otherwise double-count them. Separately, an over-tight 20-second finality window read against a single public RPC produced spurious "independent finalized RPC views did not agree" failures even though the transfer had settled safely; widening to a 40-second window with genuine dual-RPC agreement removed the false negatives without weakening the finality guarantee.

Deployment / economic constant	Value
Settlement asset	Solana Devnet USDC (exact/SVM)
Protocol	x402 v2, exact scheme, Pay.sh + MPP
Ledger unit	USDC atomic (6 decimals), JSON string
Managed rate	1 USDC = ₩1,350 (fixed test)
Price bands / top-up	₩5–₩1,000 · standalone 1–1,000 USDC
Split (policy / Devnet rail)	90/10 · all-but-1 atomic to owner
Finality	2 independent RPC origins must agree
Compute planes	Cloudflare Pages + 4 Cloud Run (asia-northeast3)
System of record	Cloud SQL PostgreSQL 16 (<code>ax-apps-db/obolus</code>)
Signer	non-exportable Ed25519 via Cloud KMS
Live status (2026-08-20)	API + gateway 100% @ current main rev

Table 1: Obolus at a glance. All amounts are Devnet test economics — they demonstrate exact quoting, settlement, recovery, payout, and refund behavior, and are not fiat, redeemable value, or a live exchange rate.

Verified live. On 2026-08-20 the API and gateway both served 100% of traffic from the revision built for the current main, and the deployed Pages build matched it. An end-to-end run on production — wallet challenge → verify → prepaid session → balance — returns 201 on the prepaid-session route across both the direct Cloud Run origin and the same-origin Pages proxy, and the ownership-reclaim path of \$5 reproduces cleanly on both. A hosted Devnet evidence run separately records real test-USDC funding, owner payout, exact remainder refund, matching finality from two RPC providers, and zero duplicate settlement.

7 Web Application

The web client is a React 19 / Vite single-page app over the same-origin API. Its center is a dialogue state machine that speaks the hit/miss branch verbatim: free search over the shelf field, an exact aggregate preview shown once, an *open with pre-paid balance* action that reserves only the quoted amount, and a Phantom top-up that signs only a buyer-chosen refill. A "Total USDC held" card reads the prepaid pot plus accrued earnings; My shelf exposes the standalone top-up rail, earnings, payouts, and multi-cycle history; and the open-calls board carries funded questions awaiting answers. Terms of the transaction render on the page rather than in a separate policy, and English and Korean are separately authored, each first-class, with Korean carrying its own type stack.

8 Artifacts and Reproducibility

Repository. One repository (`frames-clone`), main at HEAD 45105bd: a Rust/Axum backend (`store.rs`, `api.rs`) over SQLite for tests and Cloud SQL PostgreSQL 16 in production, and a React 19 / Vite single-page front over the same-origin contract-v2 API. **Deployed surfaces.** Four Cloud Run services in project `sweetspot-ax / asia-northeast3` — `obolus-api`, `obolus-gateway`, `obolus-orchestrator`, `obolus-pay` — behind a Cloudflare Pages edge at `obolus-9qi.pages.dev`; the system of record is Cloud SQL `ax-apps-db / database obolus`, the signer a non-exportable Ed25519 key in Cloud KMS, and economic intents land in a create-only Cloud Storage audit prefix.

Contracts. The twelve-endpoint contract-v2 review and settlement API plus the x402/MPP exchange (`pay curl` against a query-bound quote); authentication is a SIWX wallet challenge exchanged for an `HttpOnly, SameSite=Lax` session. **Verification.** The test floor is 221 Rust mutation survivors; a live end-to-end run on 2026-08-20 carried wallet challenge → verify → prepaid session → balance to 201 on both the direct Cloud Run origin and the same-origin Pages proxy, reproduced the \$5 ownership-reclaim path live, and a hosted Devnet evidence run recorded real test-USDC funding, owner payout, exact remainder refund, dual-RPC finality agreement, and zero duplicate settlement. **Reproducibility.** The whole stack rebuilds from `main` and redeploys through Cloud Build → canary (0% traffic behind a `/readyz` gate) → promote; artifacts are private pending a rights and security review before Mainnet.

9 Limitations and Future Work

This is a deployed, production-shaped *Devnet* system, not a Mainnet or paid-customer launch, and the honest boundary matters more than the demo. **Known gaps.** Landing and chat still quote a document in won while every balance, board, and receipt reads in USDC — a product decision to resolve, but two conventions are visible on adjacent screens. The landing hero fails closed: the WebGL shader constructs a renderer with no guard and there is no error boundary, so a browser without usable WebGL renders / blank while every other route survives. The

seeded mock backend predates the USDC work, so mock-captured demos show the pre-USDC interface.

Deliberately not claimed. Mainnet custody, commercial settlement, passkeys, A2A/AP2 interoperability, and an externally audited on-chain escrow program; email or social login at launch; and representative market validation — fixtures are demonstrations, not customer-outcome evidence. **Gates before Mainnet or paid customers.**

1. A narrow customer PoC — 20–50 real questions, a 30–100-person expert panel — measuring answer quality, time saved, willingness to pay, and contributor retention independently.
2. Measured PoC unit economics replacing demo prices, then the chosen split bound to one exact on-chain quote and receipt.
3. Cloud Monitoring SLOs and alerts for error budget, queue age, reconciliation backlog, Vertex fallback, and RPC disagreement, with rehearsed incident response.
4. Cloud SQL HA / private IP and restore drills, KMS rotation, external rate-limit / Cloud Armor validation, and a branded custom domain before material traffic.
5. Reduce the 221 Rust mutation survivors, run RustSec in CI, and complete an independent security review of identity, money movement, and recovery.

10 Conclusion

A payment rail is only as trustworthy as the guarantees it can state and keep. Obolus settles a single human document over HTTP 402 in Devnet USDC under a template neither a browser nor an autonomous agent can rewrite, keeps all money and identity behind a deterministic Rust boundary, and holds the service key in KMS alone. The system runs deployed across a Pages edge and four Cloud Run services with independent finality, idempotent settlement, and free recovery — and its correctness was not obtained in one shot but by measuring four money-path defects and closing each with an invariant, so that a displayed balance always names value that settled on chain, owned by the account that proved the key. What remains before real customers is written down as gates, not glossed as done. The exact rail exists today; the work ahead is to earn the right to price it in something other than test tokens.